

Máquinas de Estado e Flags

Curso de Microcontroladores

O problema com 'delay()'

- 'delay()' é bastante utilizado quando precisamos esperar para fazer uma ação.
- Porém, 'delay()' bloqueia o 'loop()' e paralisa o programa.
- Com o programa paralisado, você perde o controle sobre variáveis, funções e etc.
- Uma máquina de estado pode deixar o 'loop()' sempre rodando, mas também mudar uma ação após um determinado tempo.
- É essencial no seguidor, pois os sensores sempre precisam estar funcionando para a tomada de decisão.

Definindo estados com 'enum'

```
enum Estado {  
    PARADO,  
    ANDANDO  
};
```

```
Estado estadoAtual = PARADO;  
bool mudou = false;
```

- 'enum' cria um tipo com nomes simbólicos.
- Muito mais legível que usar números (0, 1, 2...).
- 'mudou' é a flag que controla transições.

Como usar no 'loop()'

```
void loop() {  
  if (!mudou) {  
    if (estadoAtual == PARADO) {  
      estadoAtual = ANDANDO;  
      digitalWrite(LED_PIN, HIGH);  
      mudou = true;  
    }  
  }  
  delay(1000);  
  mudou = false;  
}
```

Entendendo a lógica

- 'if (!mudou)' verifica se já mudou de estado.
- Dentro, você executa a ação correspondente ao estado.
- 'mudou = true' marca que a transição aconteceu.
- No fim do 'loop()', 'mudou' volta a 'false' para a próxima oportunidade.

Vantagem para robôs

- O 'loop()' roda sempre, sensores são lidos continuamente.
- Estados definem claramente o que o robô está fazendo.
- Fácil adicionar novas transições e comportamentos.

Próximos passos

- Teste o código com um LED que liga/desliga.
- Adicione um terceiro estado: 'RETORNANDO'.
- Use um botão para controlar as transições de estado.
- Estude 'millis()' para temporizações sem bloquear.